Build a talk with Slidev and an AI assistant

Coding Café hands-on · LUMC · 20 October 2026

You will build a small slide deck from scratch, written in markdown, kept in git, with an AI assistant doing the fiddly work, and publish it as a website. About **60 minutes**; stop wherever you like. Tick the boxes as you go.

The loop you'll repeat: describe → let the AI change the files → look at the slides → check the diff and the numbers → commit (or undo).

> ! Red lines
>
> - **Numbers come from data, never from the AI.** If it writes a number you didn't give it, delete it.
> - **No patient data in prompts.** Use the synthetic data below, or share only column names.
> - **Read every change** before you commit it. You present it, so you own it.

0 · Before you start (5 min)

- ☐ **Node.js 22.12 or newer**: `node -v`. If it's older, install the LTS from nodejs.org.
- ☐ **VS Code**: code.visualstudio.com
- ☐ **git**: `git --version`. On a Mac, running it the first time offers to install it.
- ☐ **An AI assistant in VS Code.** Any of these works:
  - Claude Code (VS Code extension, or `claude` in a terminal)
  - GitHub Copilot Chat in *Agent* mode
- ☐ *Optional, for step 9:* a GitHub account

Make a folder to work in and open a terminal there:

```
mkdir -p ~/coding-cafe && cd ~/coding-cafe
```

1 · Create a Slidev deck (5 min)

```
npm init slidev@latest
```

- ☐ Project name: `my-talk`
- ☐ "Install and start it now using npm?" → **Yes**
- ☐ Your browser opens the **starter deck** on http://localhost:3030

Have a look around. These keys work in any Slidev deck:

Key / address	What it does
→ ← or `space`	next / previous step
`o`	overview of all slides
`d`	dark mode
`/presenter` (after the address)	presenter view: notes, next slide, timer

- ☐ Open the folder in VS Code: **File → Open Folder → `my-talk`**, then open `slides.md`. Every slide you just saw is in this one file.

2 · Write your own slides (10 min)

In `slides.md`, keep the first block (from the first `---` to the second `---`, the settings). Delete everything below it and write your own. Use your project, or this:

```
# Readmissions after the new protocol

Lab meeting · October 2026

---

## The question

Did the new discharge protocol reduce 30-day readmissions?

---

## What we did

<v-clicks>

- Four wards, 2024 to 2025
- Before vs after 2025 Q1
- Same wards, same case mix

</v-clicks>

<!--
These are speaker notes. Only you see them, in presenter view.
-->
```

Save. The browser updates by itself.

- ☐ `---` on its own line starts a **new slide**
- ☐ `<v-clicks>` reveals a list **one item per click**
- ☐ `<!-- ... -->` at the end of a slide is a **speaker note**: check it at `/presenter`
- ☐ Try a layout: put `layout: center` between two `---` lines at the top of a slide

3 · Put it in git (3 min)

Open a second terminal in VS Code (**Terminal → New Terminal**) so Slidev keeps running in the first:

```
git init
git add -A
git commit -m "My first slides"
```

- ☐ The **Source Control** panel (^⇧G) shows no changes now. From here on, it shows exactly what the AI changed.

4 · Brief the AI once: `CLAUDE.md` (3 min)

Create a file `CLAUDE.md` next to `slides.md`. Claude Code reads it at the start of every session; for Copilot, name it `.github/copilot-instructions.md`.

```
# Briefing
- This is a Slidev deck. Slides are in slides.md. Preview: npm run dev.
- One message per slide, at most ~40 words. Split rather than shrink.
- Never invent numbers. Use data/ or write TODO.
- Change only what I ask. Don't rewrite my text.
```

- ☐ Commit it: `git add -A && git commit -m "Briefing for the AI"`

5 · Let the AI tweak your slides (10 min)

Start your assistant (Claude Code panel, `claude` in the terminal, or Copilot Chat in Agent mode). **One prompt at a time.** After each one, look at the browser, then at the diff.

1. *"Put the 'What we did' slide in a two-column layout: the list on the left, a short sentence on the right."*
2. *"Use the seriph theme and dark blue #003C7D for the slide titles."*
3. *"Add a closing slide with three take-home messages. Leave the text as TODO for me to fill in."*
4. *"You are a colleague seeing this deck once. Per slide: what is unclear? Don't edit, just list."*

- ☐ After each prompt, Source Control shows **only** the changes you asked for
- ☐ If you like it: `git add -A && git commit -m "..."`
- ☐ If you don't: `git restore .` (changed files). For new files: `git clean -n` lists them, `git clean -fd` deletes them

6 · Bring in data and a chart, on a branch (12 min)

A **branch** is a safe place to try something. Your `main` stays as it is.

```
git switch -c charts
mkdir data
curl -L -o data/readmissions.csv https://raw.githubusercontent.com/LUMC-DCC/2026-10-presenting-in-the-ai-age/main/demos/data/readmissions.csv
```

(If the download doesn't work, ask the presenter or download it from the GitHub page: `demos/data/readmissions.csv`.) The data is **synthetic**: 30-day readmission rate per quarter and ward, with a new protocol from 2025 Q1.

Prompt:

> Read data/readmissions.csv. Create components/ReadmissionChart.vue: an SVG bar chart of the "All wards" rate per quarter. Bars before 2025 Q1 grey, from 2025 Q1 dark blue #003C7D. Import the CSV with Vite's ?raw import and parse it in the component. Add one new slide at the end that uses it. Don't change my other slides.

- ☐ The chart appears on a new slide
- ☐ **Check one bar against the CSV.** Does 2024 Q1 really say 18.2?
- ☐ Read the component. You'll present it, so you should be able to explain it.
- ☐ Commit: `git add -A && git commit -m "Bar chart from the data"`

Make it build up, step by step:

> Add a second chart slide: a line chart with one line per ward. All lines grey at first; each click highlights one ward in #003C7D. Use clicks: 4 on that slide.

- ☐ Click through it. Does each step say one thing?
- ☐ Commit: `git add -A && git commit -m "Ward chart, revealed on clicks"`

7 · Go back and forth with git (5 min)

```
git log --oneline --graph --all
```

- ☐ `git switch main`: the chart slides **disappear** from the browser
- ☐ `git switch charts`: they're **back** (if Slidev shows an error after switching, press `r` in its terminal to restart it)

- ☐ **Try two versions:** `git switch -c charts-v2 main`, then give the chart prompt again, but "as a dot plot". Compare: `git diff charts charts-v2 --stat`
- ☐ **Keep the one you like:** `git switch main`, then `git merge charts` (or `charts-v2`)

8 · Present and export (3 min)

- ☐ Present: http://localhost:3030/presenter on your laptop, the normal address on the projector
- ☐ **PDF:** `npm run export`. The first time, it asks to install `playwright-chromium`: say yes. You get `slides-export.pdf`.
- ☐ **Website:** `npm run build` writes a static site to `dist/`

9 · Publish on GitHub Pages (optional, 10 min)

1. Create an **empty** repository on GitHub (no README), for example `my-talk`.
2. Add the deploy workflow from today's repository:

```
mkdir -p .github/workflows
curl -L -o .github/workflows/deploy.yml https://raw.githubusercontent.com/LUMC-DCC/2026-10-presenting-in-the-ai-age/main/.github/workflows/deploy.yml
git add -A && git commit -m "Deploy to GitHub Pages"
```

3. Push (GitHub shows the exact commands on the new repository's page):

```
git remote add origin git@github.com:YOUR-NAME/my-talk.git
git push -u origin main
```

4. On GitHub: **Settings → Pages → Source: GitHub Actions**. Then **Actions** tab: wait for green (re-run the job if it ran before you changed the setting).
5. Your slides are at `https://YOUR-NAME.github.io/my-talk/`. Every push updates them.

Stretch goals

- ☐ **Let the AI see your running deck.** Slidev's dev server includes an MCP endpoint: `claude mcp add --transport http slidev http://localhost:3030/__mcp`. Then: *"Go through my deck slide by slide, open each one in the browser, and tell me which slide could go."*
- ☐ **Run code on a slide:** a code block marked ```` ```ts {monaco-run} ```` becomes editable and runnable.
- ☐ **Your own figure:** ask the AI to rebuild a chart you'll present soon. Share the script and the column names, not the data.
- ☐ **R or Python instead?** The same loop works with Quarto: see `demos/` in today's repository.

Cheat sheet

Slidev	
New slide	`---` on its own line
Slide settings	`layout: center`, `layout: two-cols` (with `::right::`), `class: text-center`
Reveal on click	`<v-click>…</v-click>`, `<v-clicks>` around a list
Speaker notes	`<!-- ... -->` at the end of the slide
Code, highlighted step by step	write the line ranges after the language, separated by `|`: lines 1–2, then 4, then all
Docs	sli.dev

git	
Save a snapshot	`git add -A` then `git commit -m "what changed"`
What changed?	`git status`, `git diff` (or the Source Control panel)
Undo uncommitted changes	`git restore .`
New branch / switch	`git switch -c name` / `git switch name`
History	`git log --oneline --graph --all`
Combine	`git switch main` then `git merge name`

Slides and materials: https://lumc-dcc.github.io/2026-10-presenting-in-the-ai-age/ · all Coding Café sessions: https://lumc-dcc.github.io/research_software_training/